

Parallel Computing Algorithms to Numerically Solve Poisson Equation

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology (B.Tech) in Electrical Engineering

by

Soumya Ranjan Das
22JE0971

Under the guidance of
Prof. Sethupathy S

Department of Electrical Engineering
IIT(ISM), Dhanbad

May 2026

CERTIFICATE

This is to certify that the project titled “**Parallel Implementation of Iterative Methods for Poisson’s Equation on Multi-Core Architectures**” submitted by

- Soumya Ranjan Das (22JE0971)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electrical Engineering at IIT (ISM) Dhanbad under my supervision and guidance.

The results embodied in this report have not been submitted to any other university or institute for the award of any degree.

Prof. Sethupathy S

Assistant Professor

Department of Electrical Engineering

IIT (ISM) Dhanbad

Place: Dhanbad

Date: 05-05-2022

DECLARATION

I hereby declare that the project titled “**Parallel Computing Algorithms to numerically solve poisson equation**” is an original work carried out by me under the guidance of Prof. Sethupathy S, Department of Electrical Engineering, IIT (ISM) Dhanbad.

I further declare that this work has not been submitted elsewhere for the award of any degree or diploma.

- Soumya Ranjan Das (22JE0971)

Place: Dhanbad

Date: 05-05-2026

ACKNOWLEDGEMENT

I express our sincere gratitude to my guide, Prof. Sethupathy S, Assistant Professor, Department of Electrical Engineering, IIT (ISM) Dhanbad, for his continuous guidance, valuable suggestions, and encouragement throughout the course of this project.

I would also like to thank the faculty members of the Department of Electrical Engineering for providing us with the necessary academic foundation and resources required for completing this work.

I am grateful to our friends and peers for their support, discussions, and constructive feedback during the development of this project.

Finally, I would like to thank my family for their constant support and motivation.

The Author

ABSTRACT

The Poisson equation is a fundamental partial differential equation widely used in various engineering and scientific applications, including electrostatics, heat transfer, and fluid dynamics. Solving this equation numerically becomes computationally expensive for large-scale problems, especially in higher dimensions. This project focuses on accelerating the numerical solution of the Poisson equation using parallel computing techniques.

The equation is discretized using the finite difference method and solved using the Jacobi iterative method for one-dimensional (1D), two-dimensional (2D), and three-dimensional (3D) domains. To improve computational efficiency, parallel implementations are developed using three different paradigms: OpenMP for shared-memory systems, CUDA for GPU-based parallelism, and MPI with one-sided communication for distributed-memory systems.

A comparative analysis is performed between serial and parallel implementations to evaluate performance improvements in terms of execution time, speedup, and scalability. The results demonstrate that parallel computing significantly reduces computation time, with CUDA achieving the highest performance due to massive thread-level parallelism, followed by OpenMP and MPI implementations.

This study highlights the effectiveness of parallel computing in solving large-scale numerical problems and provides insights into the advantages and limitations of different parallel architectures. The work also establishes a foundation for further research into more advanced numerical solvers and hybrid parallel computing approaches.

Contents

CERTIFICATE	1
DECLARATION	2
ACKNOWLEDGEMENT	3
ABSTRACT	4
LIST OF FIGURES	7
1 INTRODUCTION	8
1.1 Background and Motivation	8
1.2 Problem Statement	8
1.3 Objectives	8
1.4 Scope of the Project	8
1.5 Organization of the Report	9
2 LITERATURE REVIEW	10
2.1 Overview of Existing Work	10
2.2 Research Gaps Identified	10
2.3 Summary	10
3 METHODOLOGY	11
3.1 Mathematical Modelling	11
3.2 Architecture	12
3.3 Implementation Details	12
3.3.1 Numerical Algorithm	12
3.3.2 Serial Code Implementation	13
3.3.3 OpenMP Implementation	13
3.3.4 CUDA C Basics	17
3.3.5 Implementation in CUDA C	21
3.3.6 MPI One-Sided Communication (RMA)	25
3.3.7 Implementation in OpenMPI	27
4 RESULTS AND ANALYSIS	31
5 CONCLUSION AND FUTURE SCOPE	34
5.1 Summary of Work	34
5.2 Key Findings	34
5.3 Limitations	34

5.4	Scope for Future Work	35
-----	---------------------------------	----

List of Figures

3.1	Flowchart of Jacobi Iterative Method	13
3.2	Splitting among the threads	14
3.3	Domain Splitting among the threads	17
3.4	Grid, block and thread organization	18
3.5	architecture	22
4.1	seriel code 1d	31
4.2	seriel code 2d	31
4.3	seriel code 3d	32
4.4	cuda	32
4.5	openmp (1D and 2D)	32
4.6	openmpi 1D	33

Chapter 1 INTRODUCTION

1.1 Background and Motivation

The Poisson equation is a fundamental partial differential equation that arises in a wide range of engineering and scientific applications, including electrostatics, heat transfer, fluid dynamics, and semiconductor device modeling. Its numerical solution is essential when analytical solutions are not feasible, especially for complex geometries and boundary conditions.

With the increasing size of computational domains, especially in two-dimensional (2D) and three-dimensional (3D) problems, the computational cost of solving the Poisson equation becomes significantly high. Traditional serial algorithms are often inefficient for large-scale problems due to their high execution time.

To address this challenge, parallel computing techniques have emerged as a powerful solution. By distributing the computational workload across multiple processing units such as CPUs and GPUs, parallel computing enables faster and more efficient numerical simulations.

1.2 Problem Statement

The numerical solution of the Poisson equation using iterative methods such as the Jacobi method requires repeated updates over a discretized grid until convergence is achieved. As the grid size increases, the number of computations grows rapidly, leading to increased execution time.

The primary challenge addressed in this project is to reduce the computational time required to solve the Poisson equation by leveraging parallel computing techniques while maintaining numerical accuracy.

1.3 Objectives

The main objectives of this project are:

- To implement the Jacobi iterative method for solving the Poisson equation in 1D, 2D, and 3D domains.
- To develop parallel implementations using OpenMP, CUDA, and MPI.
- To compare the performance of serial and parallel implementations.
- To analyze speedup and scalability across different computational architectures.

1.4 Scope of the Project

This project focuses on solving the Poisson equation using finite difference discretization and the Jacobi iterative method. Parallelization is achieved using:

- OpenMP for shared-memory CPU parallelism,
- CUDA for GPU-based parallel computing,
- MPI with one-sided communication for distributed-memory systems.

The study is limited to structured grids and does not include advanced solvers such as multigrid or conjugate gradient methods.

1.5 Organization of the Report

This report is organized as follows:

- Chapter 1 introduces the problem and motivation.
- Chapter 2 reviews existing literature and identifies research gaps.
- Chapter 3 describes the methodology, mathematical formulation, and implementations.
- Chapter 4 presents results and performance analysis.
- Chapter 5 concludes the work and discusses future scope.

Chapter 2 LITERATURE REVIEW

2.1 Overview of Existing Work

The Poisson equation has been extensively studied in the field of numerical analysis and scientific computing. Traditional approaches for solving the equation include finite difference, finite element, and finite volume methods. Among iterative solvers, the Jacobi, Gauss-Seidel, and Successive Over-Relaxation (SOR) methods are widely used due to their simplicity.

With the rise of high-performance computing, researchers have explored parallel implementations of these methods. OpenMP has been widely adopted for shared-memory systems, enabling efficient thread-level parallelism. CUDA has revolutionized computational performance by leveraging the massive parallelism of GPUs, allowing thousands of threads to execute simultaneously.

MPI-based parallelization has been a standard approach for distributed-memory systems. Recent advancements include the use of one-sided communication (Remote Memory Access), which reduces synchronization overhead and improves scalability by allowing direct memory access between processes.

2.2 Research Gaps Identified

Despite extensive research in parallel numerical methods, several challenges remain:

- Limited comparative analysis of different parallel paradigms (OpenMP, CUDA, MPI) on the same problem.
- Inefficiencies in communication overhead in distributed systems.
- Underutilization of one-sided MPI communication in iterative solvers.
- Lack of unified implementations across 1D, 2D, and 3D problems.

This project addresses these gaps by implementing and comparing multiple parallel strategies for solving the Poisson equation using a consistent numerical method.

2.3 Summary

The literature highlights the importance of parallel computing in solving large-scale numerical problems. While several methods exist, there is a need for practical implementations that compare different parallel architectures under a unified framework. This project contributes by providing such a comparative study using OpenMP, CUDA, and MPI.

Chapter 3 METHODOLOGY

3.1 Mathematical Modelling

The 1D Poisson equation is given by:

$$\frac{d^2\phi(x)}{dx^2} = -\frac{\rho(x)}{\varepsilon}$$

To solve this equation numerically, it is discretized using the finite difference method.

$$\frac{\phi(x-1) - 2\phi(x) + \phi(x+1)}{\Delta^2} = -\frac{\rho}{\varepsilon}$$

Rearranging,

$$\phi(x) = \frac{\phi(x-1) + \phi(x+1) + b}{2}$$

where Δ is the discretization step size and $b = \frac{\rho\Delta^2}{\varepsilon}$.

The solution is obtained using the Jacobi iterative method. In this approach, the neighboring terms $\phi(x-1)$ and $\phi(x+1)$ correspond to values from the previous iteration and are not updated within the current iteration.

The 2D Poisson equation is given by:

$$\frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = -\frac{\rho}{\varepsilon}$$

Applying finite difference discretization,

$$\frac{\phi(i-1, j) - 2\phi(i, j) + \phi(i+1, j)}{\Delta^2} + \frac{\phi(i, j-1) - 2\phi(i, j) + \phi(i, j+1)}{\Delta^2} = -\frac{\rho}{\varepsilon}$$

which simplifies to:

$$\phi(i, j) = \frac{\phi(i-1, j) + \phi(i+1, j) + \phi(i, j-1) + \phi(i, j+1) + b}{4}$$

where Δ is the grid spacing and $b = \frac{\rho\Delta^2}{\varepsilon}$.

All neighboring values $\phi(i-1, j)$, $\phi(i+1, j)$, $\phi(i, j-1)$, and $\phi(i, j+1)$ are taken from the previous iteration.

The 3D Poisson equation is expressed as:

$$\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} + \frac{\partial^2 \phi}{\partial z^2} = -\frac{\rho}{\varepsilon}$$

Using finite difference discretization,

$$\begin{aligned} & \frac{\phi(i-1, j, k) - 2\phi(i, j, k) + \phi(i+1, j, k)}{\Delta^2} \\ & + \frac{\phi(i, j-1, k) - 2\phi(i, j, k) + \phi(i, j+1, k)}{\Delta^2} \\ & + \frac{\phi(i, j, k-1) - 2\phi(i, j, k) + \phi(i, j, k+1)}{\Delta^2} = -\frac{\rho}{\varepsilon} \end{aligned}$$

Rewriting,

$$\phi(i, j, k) = \frac{1}{6} \left[\phi(i-1, j, k) + \phi(i+1, j, k) + \phi(i, j-1, k) + \phi(i, j+1, k) + \phi(i, j, k-1) + \phi(i, j, k+1) + b \right]$$

where Δ is the discretization step size and $b = \frac{\rho \Delta^2}{\varepsilon}$.

The neighboring terms $\phi(i-1, j, k)$, $\phi(i+1, j, k)$, $\phi(i, j-1, k)$, $\phi(i, j+1, k)$, $\phi(i, j, k-1)$, and $\phi(i, j, k+1)$ are taken from the previous iteration.

3.2 Architecture

—ADD FIGURES—

3.3 Implementation Details

This Project aims to Parallelize the Jacobi iteration using 3 different tools and report the speedup and verify the results.

- Using OpenMP
- Using CUDA C for NVIDIA GPUs
- Using OpenMPI

3.3.1 Numerical Algorithm

The Poisson equation is solved using the Jacobi iterative method. The computational domain is discretized into a uniform grid, and the governing equation is approximated using finite differences.

An initial guess for the potential ϕ is assigned over the entire domain. Boundary conditions are applied and remain fixed throughout the iterations.

At each iteration, the value of ϕ at a grid point is updated using the average of its neighboring values from the previous iteration along with the source term. The updates are performed on a separate array to ensure that only values from the previous iteration are used.

The process is repeated until the solution converges, which is determined when the maximum change between successive iterations falls below a predefined tolerance.

This method is simple to implement and highly suitable for parallelization since updates at each grid point are independent.

3.3.2 Serial Code Implementation

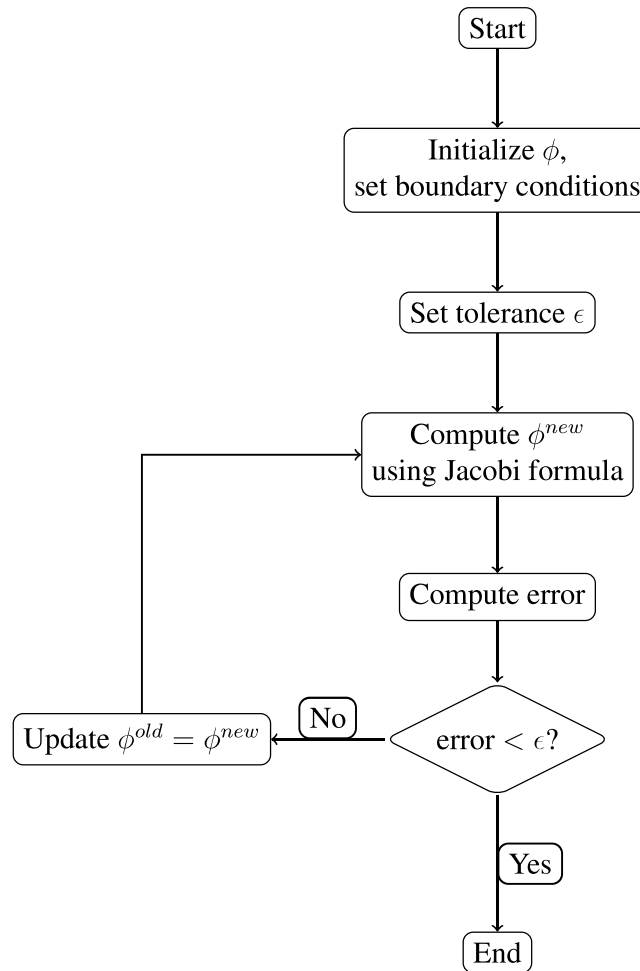


Figure 3.1: Flowchart of Jacobi Iterative Method

3.3.3 OpenMP Implementation

Introduction

OpenMP uses compiler directives (also called pragmas), library routines, and environment variables to control parallel execution.

The most common compiler directive is:

```
1 #pragma omp parallel
```

Here is a simple example in C

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      #pragma omp parallel
6      {
7          int id = omp_get_thread_num();
8          printf("Hello from thread %d\n", id);
9      }
10     return 0;
11 }
```

Here each thread prints its own ID, showing that multiple threads are running simultaneously.

A thread runs the program sequentially. Spawning multiple threads helps us to parallelly run multiple sequential instructions.

Each CPU physical core can handle usually two threads. For example, an 8-core CPU can handle 16 threads, allowing it to work on 16 tasks or sub-tasks at the exact same time.

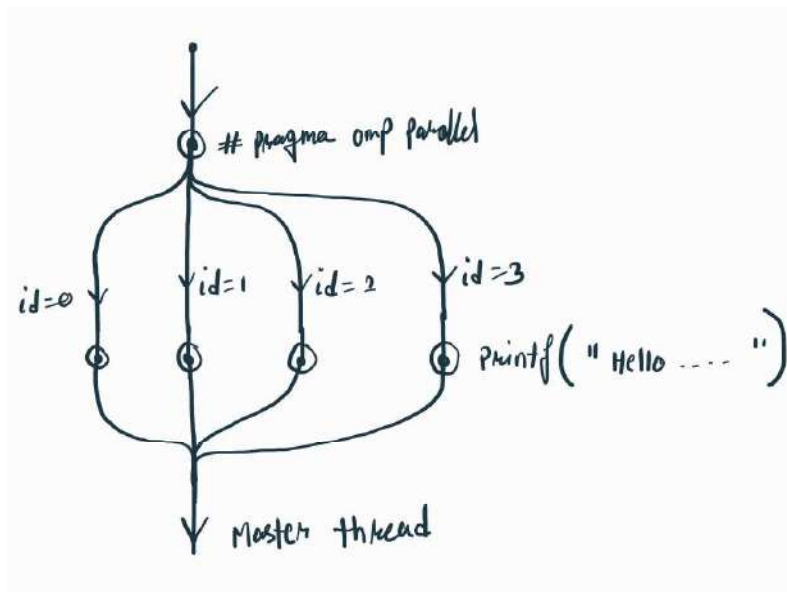


Figure 3.2: Splitting among the threads

—————[HANDWRITTEN NOTES LINK HERE](#)—————

Algorithm

For 1D Jacobi Iteration Algorithm:

1 **float*** phi_old, **float*** phi_new, **int** N are the inputs.

Declaration of Constants.

```
1     float delta = 0.2f;
2     float epsilon = 8.8f;
3     float charge_density= 1.0f; // pC/m
4
5     float b= charge_density*delta*delta/epsilon;
6
7     float left_boundary = 10.0f;
8     float right_boundary = -10.0f;
```

Parallel Region:

```
1     #pragma omp parallel
2     {
3         for(int i=0; i<iterations; i++){
4
5             #pragma omp for schedule(static,256)
6             for(int j=0; j<N; j++){
7                 float left = (j==0) ? left_boundary : phi_old[j-1];
8                 float right = (j==N-1) ? right_boundary :
                   ↪ phi_old[j+1];
9
10                 phi_new[j] = (left+right+b)/2;
11
12             }
13
14             #pragma omp single
15             {
16                 float* tmp = phi_old;
17                 phi_old = phi_new;
18                 phi_new = tmp;
19             } // This is a pointer swap. This statement is executed
               ↪ by only one thread
20
21             // Convergence
22             float max_diff = 0.0f;
23             #pragma omp for schedule(static,256)
24             for (int p = 0; p < N; p++){
25                 max_diff = fmax(max_diff, fabs(phi_new[p] -
                   ↪ phi_old[p]));
26             }
27             if (max_diff < CONVERGENCE_LIMIT) {
28
29                 break;
30             }
31         }
32     }
```


——FIGURE HOW THREAD BREAKS——

For 2D Problem similar algorithm is used. It will now have up and down boundary too. The 2D space is organized in a flat array row wise. Total size $N \times N$. See the figures in the architecture. Parallel Region:

```

1      #pragma omp parallel
2      {
3          for(int i=0; i<iterations; i++){
4
5              #pragma omp for schedule(static,256)
6              for(int y=0; y<N; y++){
7                  for(int x=0; x<N; x++){
8                      float left = (x==0) ? left_boundary :
9                      ↪ phi_old[y*N+x-1];
10                     float right = (x==N-1) ? right_boundary :
11                     ↪ phi_old[y*N+x+1];
12                     float top = (y==0) ? top_boundary :
13                     ↪ phi_old[y*N+x -N];
14                     float bottom = (y==N-1) ? bottom_boundary :
15                     ↪ phi_old[y*N+x +N];
16
17                     phi_new[y*N+x] = (left+right+top+bottom+b)/4;
18
19                 }
20             }
21
22             #pragma omp single
23             {
24                 float* tmp = phi_old;
25                 phi_old = phi_new;
26                 phi_new = tmp;
27
28                 // Convergence
29                 float max_diff = 0.0f;
30                 #pragma omp for schedule(static,256)
31                 for (int p = 0; p < N*N; p++){
32                     max_diff = fmax(max_diff, fabs(phi_new[p] -
33                     ↪ phi_old[p]));
34                 }
35                 if (max_diff < CONVERGENCE_LIMIT) {
36                     break;
37                 }
38             }
39         }
40     }

```

——FIGURE HOW THREAD BREAKS——

Similar Algorithm is used to solve 3D problem as well.

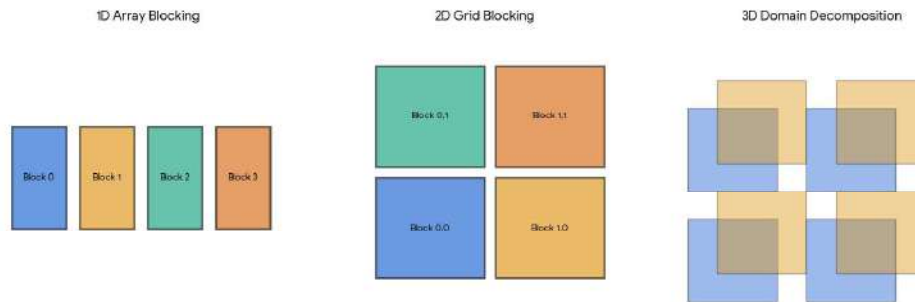


Figure 3.3: Domain Splitting among the threads

3.3.4 CUDA C Basics

Grid, Blocks and Threads

CUDA follows a hierarchical execution model:

- **Grid** = collection of blocks
- **Block** = collection of threads
- **Thread** = smallest execution unit

A kernel is launched over a grid of blocks:

```
1 kernel<<<numBlocks, threadsPerBlock>>>(parameters);
```

Key points:

- Threads execute the same code in parallel
- Blocks are independent
- Warps are groups of threads scheduled together

Grids and blocks can be 1D, 2D, or 3D.

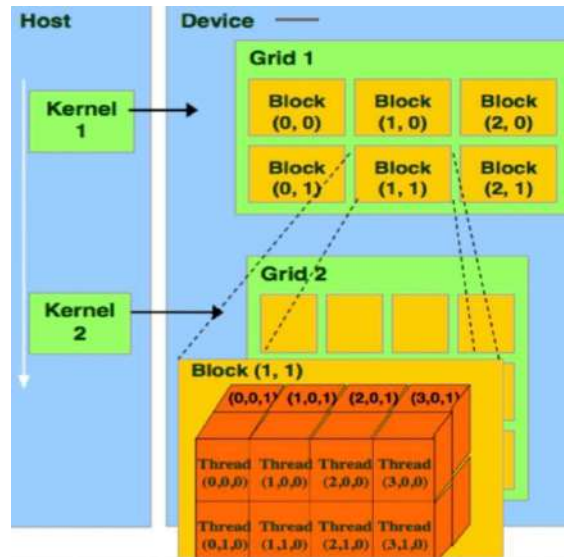


Figure 3.4: Grid, block and thread organization

Example of Grid Organization

Suppose:

- Grid size = 2×3
- Block size = 4×4

Total threads:

$$(2 \times 3) \times (4 \times 4) = 96$$

Each thread executes the same kernel but works on different data.

Kernel Declaration

CUDA kernels are declared using `__global__`:

```

1 __global__ void add(int *a, int *b, int *c)
2 {
3     int idx = threadIdx.x;
4     c[idx] = a[idx] + b[idx];
5 }
```

Kernel Launch

```

1 dim3 blocks(N/Tile_width, N/Tile_width); // This datatype can be
  ↳ used for multidimensional block allocation
2 add<<<numBlocks, threadsPerBlock>>>(a, b, c);
```

This launches multiple copies of the kernel in parallel.

Important:

- Each thread runs the same function
- Execution happens on GPU

Thread Indexing

CUDA provides built-in variables:

- `threadIdx.x`, `threadIdx.y`
- `blockIdx.x`, `blockIdx.y`
- `blockDim.x`
- `gridDim.x`

Global Index Calculation :

```
1   int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

This ensures each thread accesses a unique element.

Note: Maximum threads per block is typically 1024.

Device Memory Allocation

Memory on GPU is allocated using:

```
1  int *d_a;  
2  cudaMalloc((void**)&d_a, N * sizeof(int));
```

Free memory:

```
1  cudaFree(d_a);
```

This can be done from host side. d_a is the pointer in host memory that contains the address of device memory. This can be directly pass to a kernel.

But if you want to print the contents of this pointer then it cannot be done in host side . We need to copy the contents of the device memory to a host memory (allocated using malloc).

Or if you want to initialize a memory in host and then copy it to the device memory.

Memory Copy

```
1  cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);  
2  cudaMemcpy(h_a, d_a, size, cudaMemcpyDeviceToHost);
```

In both cases you first need to allocate the required memory in host or device side then only you can manipulate or copy the contents.

Important:

- Device memory cannot be accessed directly by CPU
- Data must be explicitly copied

Pointer Concepts

- Device pointers reside in GPU memory
- Host pointers reside in CPU memory
- We cannot dereference device pointers on CPU

2D Array Allocation

Host side:

```
1 float **h_A;  
2 h_A = (float**)malloc(N * sizeof(float*));
```

Device side:

```
1 float **d_A;  
2 cudaMalloc((void**)&d_A, N * sizeof(float*));
```

Each row must be allocated separately.

Memory Initialization

- GPU memory cannot be initialized directly from host
- Initialization must be:
 - Done on CPU, then copied OR
 - Done inside a kernel

Workflow

1. Allocate host memory
2. Initialize host data
3. Allocate device memory
4. Copy host → device
5. Execute kernel
6. Copy device → host

Barrier Synchronization

```
1 __syncthreads();
```

Purpose:

- Ensures all threads in a block reach this point
- Used when sharing data via shared memory

Measuring Performance using CUDA Events

Event Creation:

```
1 cudaEvent_t start, stop;
2
3 cudaEventCreate(&start);
4 cudaEventCreate(&stop);
```

Recording Events:

```
1 cudaEventRecord(start, 0);
2
3 // Kernel execution
4
5 cudaEventRecord(stop, 0);
6 cudaEventSynchronize(stop);
```

Elapsed Time:

```
1 float time;
2 cudaEventElapsedTime(&time, start, stop);
```

Cleanup

```
1 cudaEventDestroy(start);
2 cudaEventDestroy(stop);
```

Summary

- CUDA uses grid → block → thread hierarchy
- Kernels are launched using `<<< numBlocks, threadsPerBlock >>>`
- Memory must be explicitly managed
- Indexing is crucial for correctness
- Shared memory improves performance
- CUDA events help measure execution time

——LINK TO HANDWRITTEN NOTES——

3.3.5 Implementation in CUDA C

Introduction

CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA that allows execution of programs on the GPU. Unlike OpenMP, which utilizes CPU threads, CUDA exploits massive parallelism by launching thousands of lightweight GPU threads.

In CUDA, the CPU is referred to as the **host** and the GPU is referred to as the **device**. The execution flow involves:

1. Allocating memory on the GPU
2. Copying data from host to device
3. Launching the kernel (parallel function)
4. Copying results back to the host

The parallel execution model follows a hierarchy:

- Grid $\rightarrow$ Blocks $\rightarrow$ Threads

Each thread executes the same kernel but operates on different data.

Computation Architecture

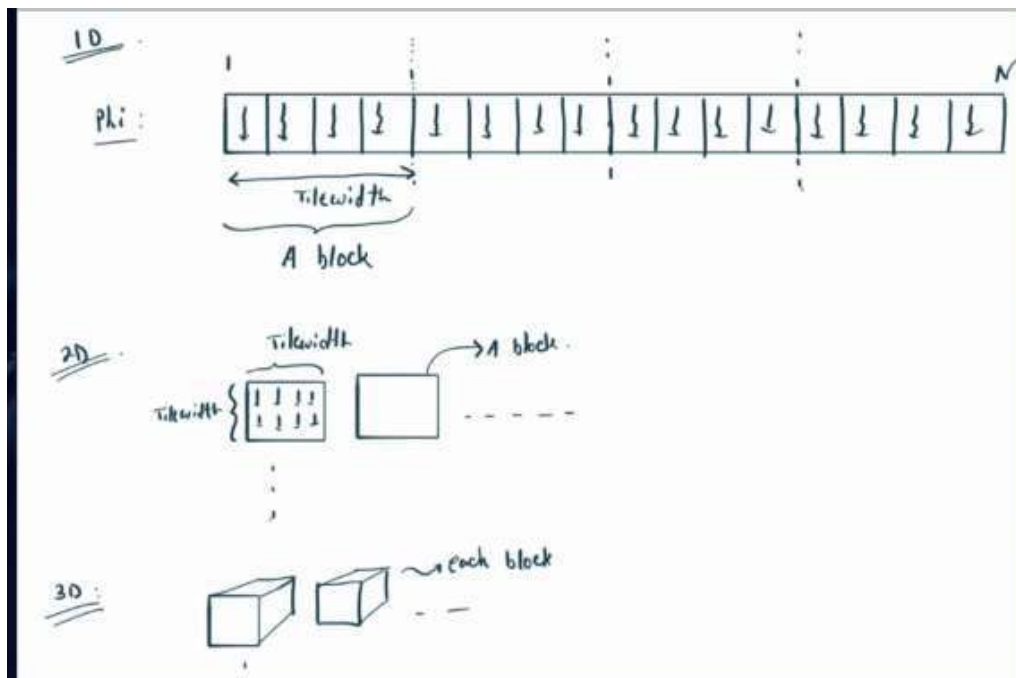


Figure 3.5: architecture

Kernel Design

The Jacobi iteration is implemented as a CUDA kernel where each thread is responsible for updating one grid point.

```

1  __global__ void jacobi_kernel(float* phi_old, float* phi_new, int N,
   ↪ float b) {
2
3      int idx = blockIdx.x * blockDim.x + threadIdx.x;
4
5      if (idx < N) {
6          float left = (idx == 0) ? left_boundary : phi_old[idx - 1];
7          float right = (idx == N-1) ? right_boundary : phi_old[idx +
   ↪ 1];

```

```

8
9         phi_new[idx] = (left + right + b) / 2.0f;
10    }
11 }

```

Each thread computes one element independently, making the algorithm highly parallelizable.

Kernel Launch Configuration

The kernel is launched using the triple chevron syntax:

```

1  int threadsPerBlock = 256;
2  int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;
3
4  jacobi_kernel<<<numBlocks, threadsPerBlock>>>(d_phi_old, d_phi_new,
    ↪  N, b);

```

Explanation:

- Threads are grouped into blocks
- Blocks form a grid
- Total threads $\geq N$ to cover all elements

Memory Allocation

Memory is explicitly managed in CUDA.

```

1  float *d_phi_old, *d_phi_new;
2
3  cudaMalloc((void**)&d_phi_old, N * sizeof(float));
4  cudaMalloc((void**)&d_phi_new, N * sizeof(float));

```

Important:

- Device memory cannot be accessed directly by CPU
- All operations must be done through kernels or memory copies

Memory Transfer

```

1  cudaMemcpy(d_phi_old, h_phi_old, N * sizeof(float),
    ↪  cudaMemcpyHostToDevice);

```

After computation:

```

1  cudaMemcpy(h_phi_old, d_phi_old, N * sizeof(float),
    ↪  cudaMemcpyDeviceToHost);

```


Iteration and Pointer Swapping

Similar to OpenMP, Jacobi iteration uses two arrays.

```
1  for(int iter = 0; iter < iterations; iter++) {
2
3      jacobi_kernel<<<numBlocks, threadsPerBlock>>>(d_phi_old,
4          ↪ d_phi_new, N, b);
5
6      float* temp = d_phi_old;
7      d_phi_old = d_phi_new;
8      d_phi_new = temp;
9  }
```

This avoids overwriting values within the same iteration.

2D Implementation

For a 2D grid, threads are organized in 2D blocks.

```
1  __global__ void jacobi_2d(float* phi_old, float* phi_new, int N,
2      ↪ float b) {
3
4      int x = blockIdx.x * blockDim.x + threadIdx.x;
5      int y = blockIdx.y * blockDim.y + threadIdx.y;
6
7      if (x < N && y < N) {
8
9          int idx = y * N + x;
10
11         float left  = (x == 0) ? left_boundary   : phi_old[idx -
12             ↪ 1];
13         float right = (x == N-1) ? right_boundary : phi_old[idx +
14             ↪ 1];
15         float top    = (y == 0) ? top_boundary    : phi_old[idx -
16             ↪ N];
17         float bottom = (y == N-1) ? bottom_boundary : phi_old[idx +
18             ↪ N];
19
20         phi_new[idx] = (left + right + top + bottom + b) / 4.0f;
21     }
22 }
```

Kernel launch:

```
1  dim3 threadsPerBlock(16,16);
2  dim3 numBlocks((N+15)/16, (N+15)/16);
3
4  jacobi_2d<<<numBlocks, threadsPerBlock>>>(d_phi_old, d_phi_new, N,
5      ↪ b);
```

Synchronization

CUDA kernels are asynchronous. To ensure completion:

```
1 cudaDeviceSynchronize();
```

This ensures that all GPU computations are finished before proceeding.

Performance Measurement

CUDA provides event-based timing.

```
1 cudaEvent_t start, stop;
2 cudaEventCreate(&start);
3 cudaEventCreate(&stop);
4
5 cudaEventRecord(start);
6
7 jacobi_kernel<<<numBlocks, threadsPerBlock>>>(...);
8
9 cudaEventRecord(stop);
10 cudaEventSynchronize(stop);
11
12 float time;
13 cudaEventElapsedTime(&time, start, stop);
```

3.3.6 MPI One-Sided Communication (RMA)

The Idea

In traditional MPI communication, two processes must cooperate: one sends and the other receives. Both must be ready at the same time.

This is called **two-sided communication**.

But this creates a problem — what if one process is slow?

Then the other process must wait.

MPI introduces a different idea: **What if one process could directly access another process's memory?**

No waiting. No coordination.

This is called **one-sided communication** or **Remote Memory Access (RMA)**.

As described in the notes :contentReference[oaicite:0]index=0, each process exposes a part of its memory, and other processes can read or write to it without explicit synchronization.

Key intuition:

Instead of asking for data, you just go and take it.

Two-Sided vs One-Sided

Two-sided model:

- Sender calls `MPI_Send`
- Receiver calls `MPI_Recv`

- Both must participate
- Delay in one affects the other

One-sided model:

- Only one process initiates communication
- No need for remote process involvement
- More asynchronous and flexible

The Concept of a Window

By default, memory in MPI is private.

To allow remote access, a process must explicitly expose memory.

This exposed region is called a **window**.

- A window defines memory accessible by other processes
- All processes in a communicator share this window

Analogy:

The process is a house. A window is literally a window others can look through or pass things into.

Creating a Window

There are multiple ways to create a window:

- `MPI_Win_create` → expose existing memory
- `MPI_Win_allocate` → allocate + expose memory

```
MPI_Win_create(base, size, disp_unit, info, comm, &win);
```

Here:

- `base` → pointer to memory
- `size` → size of memory
- `win` → window object

Core Operations

Once memory is exposed, three main operations exist:

- **MPI_Put** → write to remote memory
- **MPI_Get** → read from remote memory
- **MPI_Accumulate** → update remote memory

Important:

- These operations are **non-blocking**
- They do not immediately guarantee completion

Synchronization

Even though communication is one-sided, correctness still needs synchronization.

You must ensure:

- Data has arrived before using it

This is done using window synchronization mechanisms.

Key idea:

Data movement is decoupled from synchronization.

This is the most important concept in RMA.

- Two-sided MPI → conversation (both must talk)
- One-sided MPI → shared board (just read/write)

RMA turns communication into memory access.

—LINK TO LEARNING RESOURCES FOR MPI—

3.3.7 Implementation in OpenMPI

MPI Implementation (One-Sided Jacobi)

The Story of Execution

In CUDA, thousands of threads worked independently on a GPU.

In MPI, there are no threads like that.

Instead, we have **multiple processes**, each with its own memory.

So the question becomes:

How do multiple independent processes collaborate to solve one problem?

The answer in this project is: **Divide the domain, and let each process own a part of the grid.**

Each process is responsible for updating only its portion of the solution.

But there is a catch.

Each grid point depends on its neighbors — even neighbors that belong to other processes.

This is where communication becomes necessary.

From Communication to Memory Access

Instead of using traditional `MPI_Send` and `MPI_Recv`, we use **one-sided communication (RMA)**.

Each process exposes its memory, and others can directly read or write into it.

We do not "send" boundary values. We simply read them from the neighbor's memory.

Step 1: Domain Decomposition

The global grid is divided among processes.

- Each process gets a chunk of the domain
- Only local indices are updated
- Boundary values depend on neighboring processes

So each process has:

- Local grid (its own data)
- Ghost cells (to store neighbor data)

Step 2: Creating a Window

Each process exposes its local array using an MPI window.

```
MPI_Win_create(phi_old, local_size*sizeof(float),
               sizeof(float), MPI_INFO_NULL,
               MPI_COMM_WORLD, &win);
```

Now:

Every process can directly access the memory of every other process.

Step 3: Iterative Jacobi Update

The computation proceeds iteratively.

At each iteration:

1. Fetch boundary values from neighbors
2. Update local grid points
3. Swap arrays

Key idea:

Computation is local, but boundaries come from remote memory.

Step 4: Remote Memory Access

Instead of exchanging messages, we use:

- `MPI_Get` → read neighbor values

Example:

```
MPI_Get(&left_ghost, 1, MPI_FLOAT, left_rank,
        remote_index, 1, MPI_FLOAT, win);
```

This directly reads data from the neighbor's memory.

Step 5: Synchronization

Even though communication is one-sided, we must ensure correctness.

```
MPI_Win_fence(0, win);
```

This ensures:

- All remote accesses are completed
- Data is consistent before computation

Important insight:

MPI RMA separates data movement from synchronization.

Step 6: Jacobi Update Rule

Each process updates its local grid using:

```
for (int i = 0; i < local_size; i++) {  
  
    float left  = (i == 0) ? left_ghost  : phi_old[i - 1];  
    float right = (i == local_size - 1) ? right_ghost : phi_old[i + 1];  
  
    phi_new[i] = (left + right + b) / 2.0;  
}
```

Interpretation:

- Interior points use local data
- Boundary points use ghost values from neighbors

Step 7: Pointer Swapping

Like CUDA, we avoid overwriting values:

```
float* temp = phi_old;  
phi_old = phi_new;  
phi_new = temp;
```

Step 8: Iteration Flow

Putting everything together:

1. Start iteration
 2. Synchronize window
 3. Fetch boundary data using `MPI_Get`
 4. Synchronize again
 5. Perform local computation
 6. Swap arrays
- CUDA → many threads, shared memory
 - OpenMP → threads, shared memory
 - MPI → processes, distributed memory

But with RMA:

MPI behaves like shared memory — but only where you allow it.

Key Takeaway:

Each process solves its part, and quietly reads what it needs from others.

Chapter 4 RESULTS AND ANALYSIS

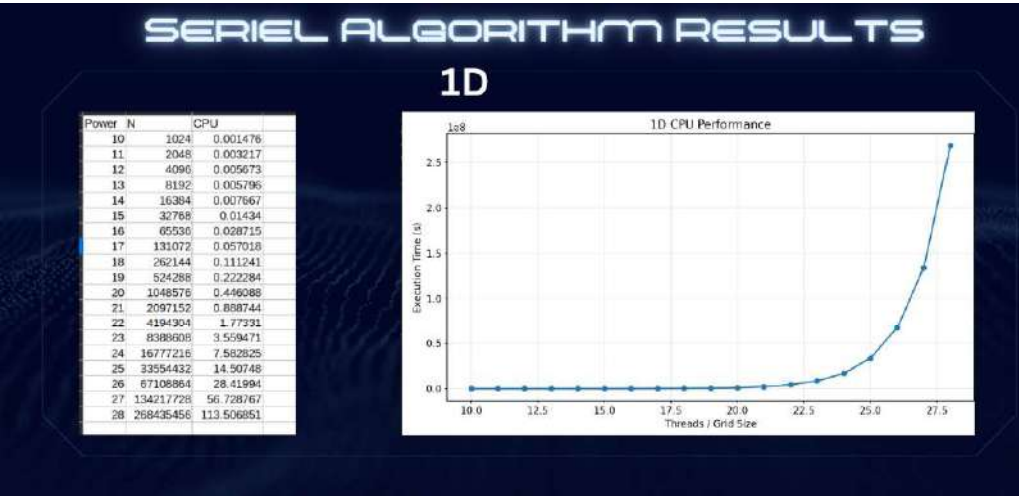


Figure 4.1: serial code 1d

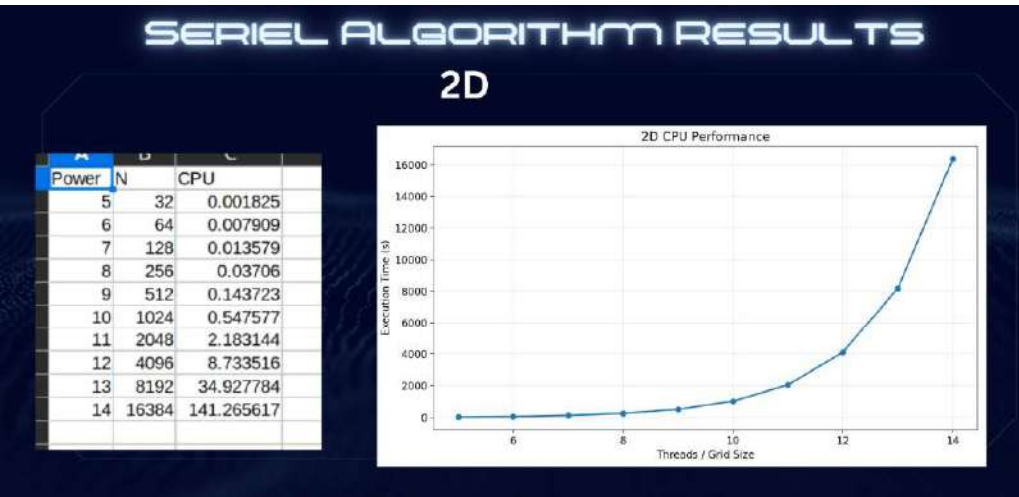


Figure 4.2: serial code 2d

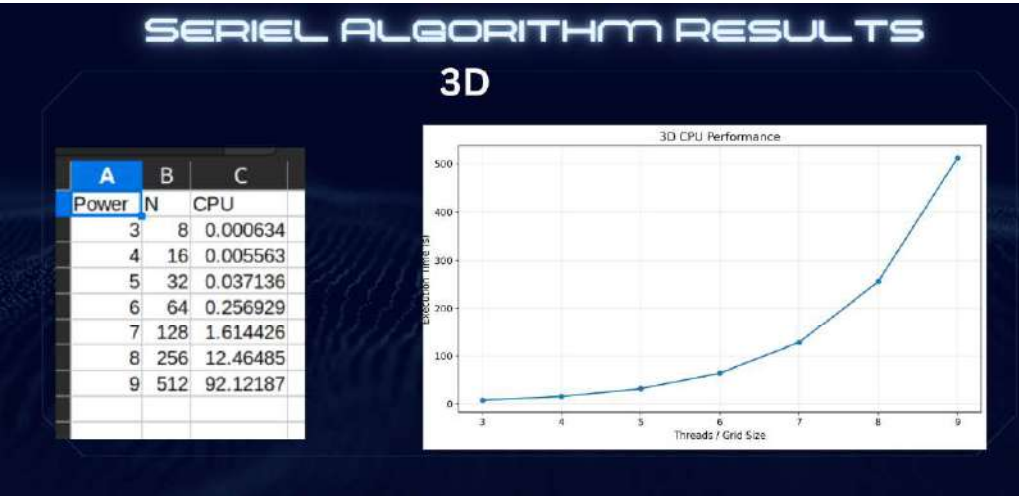


Figure 4.3: serial code 3d

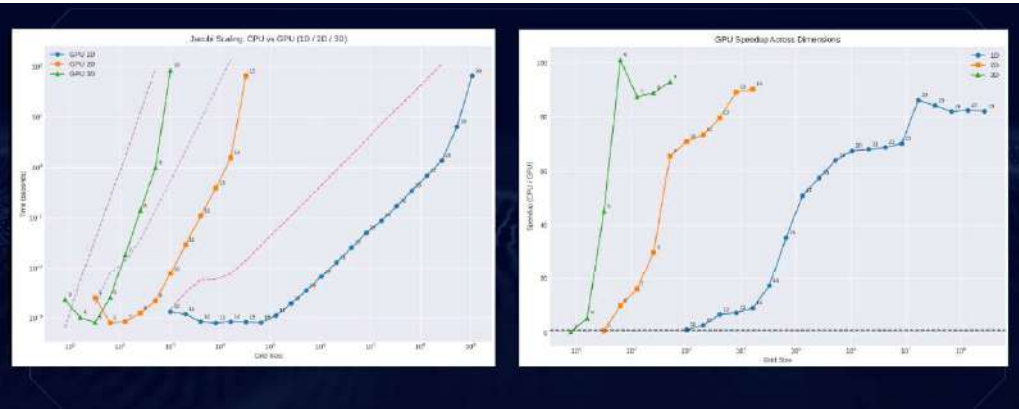


Figure 4.4: cuda

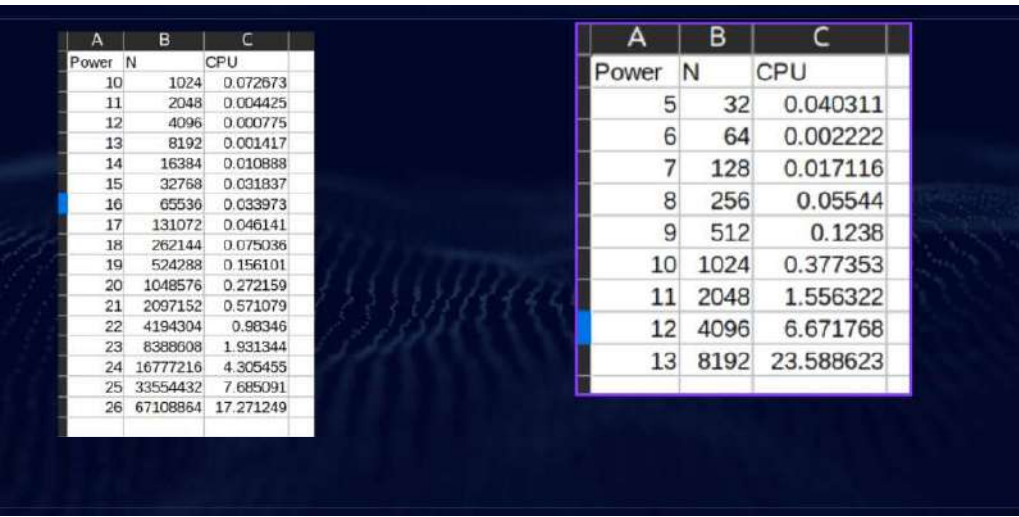


Figure 4.5: openmp (1D and 2D)

Power	N	CPU	
10	1024	0.023688	
11	2048	0.023092	
12	4096	0.020081	
13	8192	0.027279	
14	16384	0.028977	
15	32768	0.032521	
16	65536	0.033285	
17	131072	0.048719	
18	262144	0.07252	
19	524288	0.160198	
20	1048576	0.235787	
21	2097152	0.464714	
22	4194304	0.924008	
23	8388608	1.834764	
24	16777216	3.449371	
25	33554432	7.087227	
26	67108864	13.512815	
27	134217728	27.181168	
28	268435456	55.065114	

Figure 4.6: openmpi 1D

Chapter 5 CONCLUSION AND FUTURE SCOPE

5.1 Summary of Work

In this project, the Poisson equation was solved numerically using the finite difference method and the Jacobi iterative technique. The implementation was carried out in three different parallel computing paradigms: OpenMP for shared-memory systems, CUDA for GPU acceleration, and MPI with one-sided communication for distributed systems.

The problem was extended from 1D to 2D and 3D domains, and the correctness of the numerical solution was verified across all implementations.

5.2 Key Findings

The key observations from this study are:

- Parallel implementations significantly reduce execution time compared to serial computation.
- OpenMP provides a simple and effective way to parallelize computations on multi-core CPUs.
- CUDA achieves the highest performance due to massive parallelism offered by GPUs.
- MPI with one-sided communication reduces synchronization overhead and provides flexibility in distributed environments.
- The Jacobi method is highly suitable for parallelization due to its independent update structure.

5.3 Limitations

Despite the improvements, the following limitations exist:

- The Jacobi method has relatively slow convergence compared to advanced iterative methods.
- Communication overhead in MPI can still impact performance for very large systems.
- The implementation is limited to structured grids and does not consider irregular domains.

5.4 Scope for Future Work

Future work can focus on:

- Implementing faster iterative methods such as Gauss-Seidel, SOR, or Multigrid methods.
- Extending the solver to unstructured meshes and complex geometries.
- Optimizing memory access patterns in CUDA for better performance.
- Exploring hybrid parallel models combining MPI with OpenMP or CUDA.
- Applying the solver to real-world engineering problems such as electromagnetic simulations and fluid dynamics.

Bibliography

- [1] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*, Cambridge University Press, 2007.
- [2] G. D. Smith, *Numerical Solution of Partial Differential Equations: Finite Difference Methods*, Oxford University Press, 1985.
- [3] S. J. Min, *OpenMP Tutorial*, ECE 563: Programming Parallel Machines, Purdue University. :contentReference[oaicite:0]index=0
- [4] W. Gropp, *One-Sided Communication in MPI*, University of Illinois. :contentReference[oaicite:1]index=1
- [5] MPI Forum, *MPI: A Message-Passing Interface Standard*, Version 3.1, 2015.
- [6] NVIDIA Corporation, *CUDA C Programming Guide*, Available at: <https://docs.nvidia.com/cuda/>
- [7] NVIDIA Corporation, *CUDA C Best Practices Guide*, Available at: <https://docs.nvidia.com/cuda/>
- [8] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Available at: <https://www.openmp.org>
- [9] R. L. Burden and J. D. Faires, *Numerical Analysis*, Cengage Learning, 2011.
- [10] NVIDIA Developer, *CUDA Programming Tutorial*, YouTube, Available at: <https://www.youtube.com/watch?v=F620ommtjqk>
- [11] Lawrence Livermore National Laboratory, *Introduction to OpenMP*, YouTube, Available at: <https://www.youtube.com/watch?v=2f4aZtFf7G0>
- [12] Argonne National Laboratory, *MPI Tutorial for Beginners*, YouTube, Available at: <https://www.youtube.com/watch?v=ZKf1hYQ2V8c>
- [13] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, Addison-Wesley, 2003.
- [14] J. Sanders and E. Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*, Addison-Wesley, 2010.